Project Shifting Ghost — Quantum-IDS V3.0

White Paper No. 2 — Strategic Doctrine Series

Shifting Ghost:

A Doctrine of Asymmetric Inhibition and Adversarial Poisoning

A strategic analysis prepared for defence laboratories, intelligence agencies, and Tier-1 security research institutions.

Classification:	Restricted / Technical
Architecture:	Quantum-IDS V3.0
Runtime:	Rust (Zero-Allocation) + eBPF/XDP
Compliance:	NASA Power of Ten (P10)
Target Platform:	Debian 13 (Linux 6.x kernel)
Document Series:	WP-SG-002
Status:	Final Draft

Shifting Ghost Architecture Team
Lead Architect Office — Project Quantum-IDS

This document is not a user manual. It is a strategic artefact describing offensive defensive doctrine at the kernel level.

Contents

Abstract: The End of Passive Defence

Classical intrusion detection operates within a fundamentally reactive paradigm: an adversary probes, the defender observes; the adversary attacks, the defender logs; the adversary pivots, the defender reports. This model presupposes a structural asymmetry in which the cost of offence is systematically lower than the cost of defence. *Shifting Ghost* is a formal repudiation of this assumption.

Built on a Rust runtime enforcing zero dynamic allocation after initialisation and a correlated eBPF pipeline operating at the Linux kernel's innermost observation layer, Shifting Ghost inverts the cost calculus of engagement. Through a five-stage doctrine — **Reject**, **Exhaust**, **Poison**, **Confuse**, **Trap** — the system transforms the network perimeter from a passive boundary into an *active adversarial environment*.

The architecture achieves three properties that no firewall-centric or signature-based system can replicate simultaneously: (1) **deterministic sub-microsecond packet rejection** at the eBPF/XDP layer without userspace involvement; (2) **statistical poisoning of adversarial reconnaissance** through manipulated SSA eigenspectrum responses and fabricated TCP stack fingerprints; and (3) **persistent kernel-level adversary tagging** via eBPF hash maps that associate network flows with process-level activity across the entire attack surface.

This paper does not describe configuration procedures. It articulates *why* the architecture is a strategic liability for any sophisticated adversary, and why its formal properties — rooted in NASA P10 safety constraints and quantitative market microstructure theory — make it uniquely suited for deployment in high-assurance environments where the cost of compromise is existential.

Core Doctrine

Do not merely resist the adversary. Make the cost of attacking you so high, and the intelligence value of attacking you so low, that rational adversaries choose not to engage. Make irrational adversaries visible, exhausted, and trapped.

I. The Immutable Barrier: NASA P10 and Rust as a Foundation for Deterministic Rejection

1.1 The Strategic Value of Predictability

Before any algorithm is analysed, before any cryptographic primitive is evaluated, the most consequential property of a defensive system is its *predictability under adversarial load*. A system that performs excellently under nominal conditions but degrades under stress is not a defence — it is a mechanism that rewards patient, high-volume adversaries.

Shifting Ghost is built on two non-negotiable foundations that guarantee behavioural predictability: the **NASA Power of Ten (P10)** programming discipline and the **Rust type system**. These are not stylistic choices. They are load-bearing structural constraints whose combined effect is a runtime that an adversary cannot destabilise through resource exhaustion.

1.2 NASA Power of Ten: Determinism as a Security Property

The NASA Power of Ten rules, originally formulated for safety-critical flight software, become security properties when applied to a network defence system operating under adversarial load. The critical rules and their security implications are:

P10 Rule	Original Intent	Security Implication in Shifting Ghost
P10-1	No dynamic memory allocation after initialisation	Heap exhaustion attacks, allocator fragmentation attacks, and `OOM`-triggered path divergence are structurally impossible
P10-2	All loops must have a fixed upper bound	Denial-of-service via unbounded iteration is impossible; packet processing time has a provable worst-case
P10-5	No unchecked return values	Error paths are explicit and auditable; silent failure cannot create a false sense of security or corrupt state
P10-7	No recursion	Stack overflow attacks are structurally impossible; call depth is bounded at compile time
P10-8	`unsafe` code restricted to isolated modules	The attack surface for memory safety violations is enumerated and auditable

1.3 The Arena Allocator: A Hard Memory Perimeter

The zero-allocation invariant is enforced not by convention but by a typestate machine in `sg-arena`. At system initialisation, a single 256 MiB static arena is carved into all regions

required for the system's operational lifetime:

$$\text{ARENA\_TOTAL} = \underbrace{100{,}000 \times 20\,\text{B}}_{\text{Packet ring}} + \underbrace{512 \times 200\,\text{KB}}_{\text{Bucket pool}} + \underbrace{8{,}192 \times 32\,\text{B}}_{\text{FlowScore pool}} + \sum_i R_i \leq 256\,\text{MiB} \quad (1)$$

where R_i enumerates all remaining fixed regions (SSA history, EKF innovation ring, Bellman score history, whitelist, `nftables` rule handles, tarpit slots, and SIEM alert ring). The pre-computed budget totals approximately 144 MiB, leaving a 112 MiB overhead margin verified at compile time.

After `Arena::lock()` is called, the `ArenaInit` type is *consumed* by the Rust type system. No further allocation is syntactically expressible in the program. The typestate transition is:

$$\texttt{ArenaInit} \xrightarrow{\texttt{.lock()}} \texttt{ArenaLocked}$$

`ArenaInit` is `!Send`: it cannot be transmitted to any thread. `ArenaLocked` is `Send + Sync`: it can be read from any thread for diagnostic queries but cannot allocate. This is not a runtime check. It is a compile-time invariant enforced by the borrow checker.

Strategic Implication

An adversary attempting to exhaust memory through high-volume packet injection, SYN floods, or crafted fragmented streams cannot cause the system to allocate. The heap is empty after initialisation. There is nothing to exhaust.

1.4 Rust's Type System as a Formal Verification Layer

Beyond memory safety, the Rust type system enforces two additional properties directly relevant to adversarial robustness:

1.4.1 Flow State Transitions as a Finite Automaton

The `FlowState` enum encodes the adversary's lifecycle through the system as a compile-enforced finite automaton:

$$\texttt{New} \to \{\texttt{Established}, \texttt{Blocked}\} \to \texttt{Suspicious} \to \{\texttt{Blocked}, \texttt{Honeypotted}, \texttt{Expired}\}$$

Invalid transitions — for example, a flow attempting to revert from `Suspicious` to `New` — are compile errors, not runtime guards. An adversary who understands the state machine cannot exploit a path that does not exist in the compiled binary.

1.4.2 Zero-Copy Data Pipeline

The entire analytical pipeline from eBPF ringbuffer through SSA, entropy analysis, EKF, and Bellman scoring operates on `&PacketHeader` borrows pointing directly into arena memory. No crate downstream of `sg-arena` takes ownership of a packet. There are no copies, no serialisation steps, and no intermediate heap objects for an adversary to influence through memory pressure.

1.5 Kinetic Cost Asymmetry: The Arithmetic of Engagement

The eBPF/XDP pipeline processes packets at the NIC driver level, before the kernel networking stack is involved. For flows matching a known-bad entry in the `deny_map` (a `BPF_MAP_TYPE_LRU_HASH`), the verdict is `XDP_DROP` — the packet is discarded in hardware-adjacent code without ever touching userspace.

The cost structure this creates for an adversary is asymmetric by design:

Resource	**Defender Cost per Packet**	**Attacker Cost per Packet**
CPU cycles (XDP drop)	∼50–200 ns (BPF map lookup)	Full packet construction + transmission
Memory operations	1 hash lookup (zero alloc)	Full TCP stack traversal on attack host
Network bandwidth consumed	0 (dropped at driver)	Full frame bandwidth
State created	None (LRU map, pre-allocated)	Ephemeral connection state on attacker

For a sustained attack of 10^6 packets per second, the defender's CPU budget is on the order of $200\,\mu$s of total processing time per second. The attacker must sustain a full sending pipeline. At scale, this asymmetry is not merely operational — it is *infrastructure-atrophying*. Sustained engagement becomes economically irrational.

II. Algorithmic Poisoning: Transforming Stolen Data into a Liability through SSA-Driven Deception

2.1 The Intelligence Problem for the Adversary

A sophisticated adversary does not attack blindly. Before any active exploitation, an attacker invests significant resources in reconnaissance: fingerprinting the target OS, mapping open ports and services, measuring response latencies, characterising traffic patterns, and building a model of the target's network behaviour. This intelligence product is the foundation of their attack plan.

Shifting Ghost treats this reconnaissance phase not as a threat to be blocked, but as an *opportunity to inject deliberate misinformation into the adversary's intelligence pipeline.* The system does not merely refuse to answer — it answers *incorrectly*, with calibrated, internally consistent, but strategically misleading data.

2.2 Singular Spectrum Analysis as a Deception Engine

2.2.1 The Mathematical Foundation

SSA decomposes a time series of network observations — specifically, inter-packet arrival latencies — into orthogonal structural components by constructing and analysing the trajectory (Hankel) matrix. Given a latency series $\mathbf{x} = (x_1, x_2, \ldots, x_N)$, the trajectory matrix is:

$$\mathbf{X} = \begin{pmatrix} x_1 & x_2 & \cdots & x_K \\ x_2 & x_3 & \cdots & x_{K+1} \\ \vdots & \vdots & \ddots & \vdots \\ x_L & x_{L+1} & \cdots & x_N \end{pmatrix} \in \mathbb{R}^{L \times K} \tag{2}$$

where L is the window length and $K = N - L + 1$ is the number of columns. The singular value decomposition $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ yields:

$$\mathbf{X} = \sum_{i=1}^{d} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \tag{3}$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_d \geq 0$ are the singular values and $(\mathbf{u}_i, \mathbf{v}_i)$ are the left and right singular vector pairs.

In Shifting Ghost's threat model, the dominant eigenvalue $\lambda_1 = \sigma_1^2$ represents the primary mode of network traffic — the structural "heartbeat" of the communication pattern. Anomalies manifest as perturbations of this dominant mode. The system monitors:

$$\Delta\lambda_1^{(t)} = \frac{\lambda_1^{(t)} - \lambda_1^{(t-1)}}{\lambda_1^{(t-1)}} \tag{4}$$

If $|\Delta\lambda_1^{(t)}| > 0.15$ over fewer than 10 consecutive bucket windows, an anomaly is confirmed.

2.2.2 Deterministic Gaslighting: The Poison Mechanism

When a flow is classified as `SCAN` or elevated to the `High-Risk` tier, the system does not simply block it. Instead, the eBPF fingerprinting probe (`tcp_fp_probe`) becomes active for that flow, and the system's response enters a *deceptive eigenspace manipulation* mode.

The conceptual operation is as follows: when the adversary's tooling (Nmap, masscan, p0f, Zmap, custom implants) sends crafted packets designed to elicit characteristic responses from the target stack, Shifting Ghost intercepts the outgoing response at the kernel level via `tc` (Traffic Control) and rewrites specific TCP header fields. The adversary does not see the target's real responses — they see a curated set of contradictory signals.

Formally, let $\mathbf{R}_{\text{real}}$ denote the vector of real TCP response parameters for a given probe, and let $\mathbf{R}_{\text{spoof}}$ denote the synthesised spoofed response vector:

$$\mathbf{R}_{\text{spoof}} = \mathbf{R}_{\text{real}} + \delta, \qquad \delta = \begin{pmatrix} \delta_{\text{TTL}} \\ \delta_{\text{MSS}} \\ \delta_{\text{WinScale}} \\ \delta_{\text{TS}} \end{pmatrix} \tag{5}$$

where each δ_i is drawn from a pre-computed pseudo-random table (backed by arena-allocated fixed arrays, zero runtime allocation) designed to simulate a specific foreign OS fingerprint. The injected values are:

- **TTL**: set to 64 (Linux) or 128 (Windows) or 255 (Cisco IOS), inconsistently across probes from the same flow
- **MSS**: set to values ranging from 536 (ancient minimum) to 1460 (Ethernet standard) to 8960 (jumbo frames), mismatched with the advertised window
- **Window Scale**: inconsistent with the negotiated SYN options
- **TCP Timestamp**: advanced at incorrect rates, mismatched across segments
- **Unknown TCP Options**: dummy option bytes that no known OS emits, designed to corrupt fingerprint classifier confidence scores

The result is that the adversary's fingerprinting engine receives a response vector $\mathbf{R}_{\text{spoof}}$ that is *internally contradictory*: no real OS emits this combination of parameters. Automated tools such as `p0f` return low-confidence or null classifications. Human analysts examining the output face a target that simultaneously exhibits properties of multiple operating systems, none of which match.

2.2.3 The Intelligence Cost

A competent adversary who suspects their fingerprinting is corrupted must:

1. Re-run all reconnaissance with independent tools to cross-validate — doubling collection time

2. Suspect their own implants' telemetry if already deployed — triggering internal security reviews
3. Attempt to isolate the corruption source — a process that may take days and consume significant analyst time
4. Make operational decisions on a foundation of data they cannot trust — increasing their error rate

Each of these steps consumes adversarial resources. The intelligence product they have built is not merely worthless — it is *actively dangerous to them*: an attacker who proceeds on the basis of corrupted fingerprint data will misconfigure exploits for the wrong OS, target services that do not exist, and expose their TTPs through the pattern of their confusion.

2.3 Kalman-Filter-Based State Poisoning

2.3.1 The EKF as a Behavioural Baseline Oracle

Shifting Ghost maintains a continuous Extended Kalman Filter over the bivariate state space of CPU utilisation and packet-per-second rate:

$$\mathbf{x}_k = \begin{pmatrix} \text{CPU}_{\%} \\ \text{PPS}_k \end{pmatrix} \tag{6}$$

The EKF prediction step produces the a-priori state estimate:

$$\hat{\mathbf{x}}_{k|k-1} = F_k \hat{\mathbf{x}}_{k-1|k-1} \tag{7}$$

with prediction covariance:

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q_k \tag{8}$$

where F_k is the state transition matrix and Q_k is the process noise covariance. The innovation — the discrepancy between predicted and observed state — is:

$$\epsilon_k = z_k - H\hat{\mathbf{x}}_{k|k-1} \tag{9}$$

where z_k is the measurement vector and H is the observation matrix. An anomaly is declared when:

$$|\epsilon_k| > 2.7\,\sigma \tag{10}$$

This threshold corresponds to a false positive rate of approximately 0.7% under a Gaussian noise model, calibrated empirically against the 7-day baseline gold file.

2.3.2 Anti-Poisoning Architecture

The EKF is itself a target. An adversary aware of a Kalman-based IDS may attempt to slowly manipulate the filter's covariance matrices through a gradual, low-amplitude attack — a strategy known as *slow-drift poisoning*. Shifting Ghost defeats this through two mechanisms:

1. **Mandatory 24-hour reset**: Every 24 hours, the EKF state is discarded and re-initialised from `baseline_gold.json` — a file generated during the 7-day calibration phase and signed with HMAC/PGP. A tampered baseline file produces a signature mismatch and triggers the semi-manual operating mode, requiring operator acknowledgment before resuming automatic enforcement.
2. **Hurst persistence gate**: Before the EKF innovation is acted upon, its associated flow must pass the Hurst persistence test (see Section III). This ensures that a single anomalous measurement — which could be injected by a sophisticated adversary to probe the system's detection threshold — does not trigger an enforcement action unless the anomaly is statistically persistent over time.

2.3.3 The Deceptive Feedback Channel

Beyond anomaly detection, the EKF provides a subtle offensive capability: the system's *observable response behaviour* can be modulated to feed false information back to the adversary's own measurement apparatus. When an adversary monitors their attack's effectiveness by observing side-channel signals (server response latency, TCP retransmission rates, connection establishment times), the Adaptive Tar-Pit module (Section III) introduces calibrated, deterministic jitter that *simulates normal operational variability* — preventing the adversary from distinguishing "their attack is working" from "their attack is being absorbed."

The jitter parameters — delay distribution and dispersion — are drawn from the EKF's internal model of normal system behaviour. The adversary's measurement of the target becomes a reflection of the target's own model of itself, not of the actual impact of the attack.

III. Adversarial Exhaustion: How Shifting Ghost Forces Attackers into a State of Operational Futility

3.1 The Entropy Sentinel: Statistical Fingerprinting of the Adversary

3.1.1 Dual Entropy Detection

Shifting Ghost does not rely on signature matching. It characterises adversarial traffic through its *statistical geometry*. Two entropy measures are computed for each flow's payload:

Shannon Entropy — measuring the global unpredictability of the payload byte distribution:

$$H(X) = -\sum_{i=0}^{255} p_i \log_2 p_i \tag{11}$$

where p_i is the empirical probability of byte value i in the observed payload (limited to the first 128–256 bytes; no full payload storage).

Rényi Entropy of order 2 — measuring the concentration of the distribution, sensitive to tunneling and steganographic channels:

$$H_2(X) = -\log_2 \sum_{i=0}^{255} p_i^2 \tag{12}$$

The detection criteria are:

$$H(X) > 0.95 \ \wedge \ \text{port} \notin \text{StandardPorts} \implies \texttt{ENCRYPTED\_EXFIL} \tag{13}$$

$$H_2(X) \ll H(X) \ \wedge \ \text{protocol} \in \{\text{ICMP, DNS}\} \implies \texttt{ENCRYPTED\_TUNNEL} \tag{14}$$

A Shannon entropy near 1.0 indicates near-maximum randomness — a characteristic of encrypted or compressed data. On a non-standard port, this is a strong indicator of covert channel activity. A large gap between H and H_2 indicates that the distribution, while globally flat, has localised concentration — a signature consistent with steganographic encoding or protocol-based tunneling (DNS exfiltration, ICMP covert channels).

3.1.2 VPIN: Volume Imbalance as an Attack Vector Identifier

Borrowed from financial market microstructure theory, the Volume-Synchronised Probability of Informed Trading (VPIN) metric provides a measure of directional flow imbalance:

$$\text{VPIN} = \frac{\sum_{i=1}^{n} |V_i^{\text{in}} - V_i^{\text{out}}|}{\sum_{i=1}^{n} V_i^{\text{total}}} \tag{15}$$

computed over a sliding window of $n = 50$ volume buckets. In a financial context, high VPIN

indicates the presence of informed traders who consistently route volume in one direction. In the network security context:

- **VPIN** > 0.85 **on** $> 30\%$ **of the window**: Sustained directional imbalance, consistent with exfiltration (persistent outbound) or command-and-control polling (persistent inbound)
- **VPIN** combined with high $H(X)$: High-entropy directional flow — the primary signature of encrypted exfiltration at scale

3.1.3 Kernel-Level Adversary Tagging via eBPF Maps

Once a flow is classified as `High-Risk` through the composite Bellman score (Section 3.2), a cascade of kernel-level tagging events is initiated:

1. The source IP is written to the `deny_map` (`BPF_MAP_TYPE_LRU_HASH`), enabling XDP-layer drop for all subsequent packets from that source
2. The `FlowKey` is written to the `scan_flag_map` with an expiry timestamp, enabling the Deceptive Fingerprinting module for all connections from this source for 60 seconds
3. If correlation with `execve` or `write` syscalls is detected within a 1–3 second window (via the `execve_monitor` probe), the `fingerprint_map` is armed with `FpParams` specifying the maximum-deception response profile

The adversary is now *tagged at the kernel level*. Every packet they send receives a kernel-generated verdict before reaching any userspace handler. Every probe they issue returns curated misinformation. Every connection attempt is silently redirected to a honeypot. The network has become a hostile environment that is actively hunting them.

The critical property of this tagging mechanism is its *persistence without allocation*. The `BPF_MAP_TYPE_LRU_HASH` operates entirely within kernel memory, managed by the eBPF subsystem. Entries are evicted by the LRU policy when the map is full — but the map capacity is pre-sized for the expected adversary count. Tagging an adversary costs exactly one kernel map write. Enforcing the tag costs one hash lookup per packet.

3.2 The Composite Bellman Score and Threshold Architecture

All sub-scores are aggregated into a composite risk metric:

$$S_{\text{final}} = 0.4 \cdot H_{\text{entropy}} + 0.3 \cdot \text{VPIN} + 0.3 \cdot \epsilon_{\text{EKF}} \tag{16}$$

where:

- $H_{\text{entropy}} \in [0, 1]$ is the normalised Shannon entropy score for the flow
- $\text{VPIN} \in [0, 1]$ is the volume imbalance metric from Equation (15)
- $\epsilon_{\text{EKF}} \in [0, 1]$ is the normalised EKF innovation magnitude from Equation (9)

The weight sum is verified at compile time:

$$\underbrace{0.4}_{\text{WEIGHT\_ENTROPY}} + \underbrace{0.3}_{\text{WEIGHT\_VPIN}} + \underbrace{0.3}_{\text{WEIGHT\_EKF}} = 1.0 \pm 3\varepsilon_{\text{f32}}$$

as a `const` assertion in `sg-common`. Any codebase change that violates this identity fails compilation.

Actions are dispatched by percentile ranking against 30 days of historical score data:

$$\text{Action} = \begin{cases} \texttt{Observe} & S_{\text{final}} < P_{60} \\ \texttt{Tarpit} & P_{60} \leq S_{\text{final}} < P_{90} \\ \texttt{Drop} & S_{\text{final}} \geq P_{90} \ \wedge \ \neg\, \texttt{SCAN} \\ \texttt{Honeypot} & S_{\text{final}} \geq P_{90} \ \wedge \ \texttt{SCAN} \end{cases} \tag{17}$$

The percentile-based threshold approach is critical: unlike fixed thresholds, percentile boundaries adapt automatically to shifts in the ambient threat environment without operator intervention, while remaining robust to individual outliers.

3.3 Hurst Exponent: Persistence as the Final Gate

Before any enforcement action is triggered, the flow must pass a persistence verification. The Hurst exponent is estimated over the PPS time series using the rescaled range (R/S) method:

$$H = \lim_{n \to \infty} \frac{\log(R/S)_n}{\log n} \tag{18}$$

where R is the range of the mean-adjusted cumulative deviation and S is the standard deviation. Classification:

$$H \approx 0.5 \implies \text{Brownian noise (benign anomaly, transient)} \tag{19}$$

$$H > 0.65 \implies \text{Persistent process (structured attack, continuous threat)} \tag{20}$$

This gate serves a critical anti-evasion function. A sophisticated adversary who understands score-based detection systems may attempt to trigger false positives deliberately — injecting a brief burst of anomalous traffic to exhaust enforcement resources or establish a false baseline. The Hurst gate ensures that *only persistent, structurally coherent threats reach the enforcement layer.* A single anomalous bucket, however extreme, does not trigger a Drop or Honeypot action.

The combined gate for enforcement escalation is:

$$\text{Escalate} \iff H > 0.65 \ \wedge \ Z > 2.3 \ \wedge \ S_{\text{final}} \geq P_{90} \tag{21}$$

where $Z = (x - \mu)/\sigma$ is the Z-score computed via the Welford online estimator (zero additional allocation, $O(1)$ update).

3.4 The Adaptive Tar-Pit: Operational Exhaustion at the Protocol Level

For flows in the intermediate risk band ($P_{60} \leq S < P_{90}$), the system enters the *Adaptive Tar-Pit* mode. This is not a passive throttle. It is an active, calibrated mechanism for consuming adversarial resources.

3.4.1 TCP Window Zero Manipulation

The server's advertised TCP receive window for the flagged flow is set to zero. Under the TCP protocol, a zero-window advertisement forces the remote sender into an exponentially backing-off retransmission cycle (the *persist timer*). The adversary's sending stack is locked in a wait state, consuming connection state resources, file descriptors, and timing on the attack host. The connection appears alive from the adversary's perspective — there is no RST, no timeout, no error — but no data is transferred.

3.4.2 Traffic Control Jitter Injection

Concurrently, the `sg-tarpit` module issues a `tc` command to install a `netem` qdisc on the egress path for the flagged flow:

$$\text{Delay} = 200\,\text{ms} + \mathcal{U}(-50, +50)\,\text{ms}, \quad \text{Variation} = 25\%$$

where $\mathcal{U}(-50, +50)$ is a uniform random variable drawn from a pre-computed table in the arena. The resulting latency profile is indistinguishable from a heavily loaded server, and the adversary's timing-based probes (latency oracles, keystroke timing attacks) become unreliable.

The tar-pit state is maintained for 10–30 seconds. If the flow's score falls below P_{60}, the qdisc is removed and the flow is released. This reversibility is critical: the system is calibrated to avoid permanently impacting flows that self-correct, and to avoid revealing its detection capabilities through permanent behavioural changes.

3.4.3 The Asymmetric Resource Exchange

The resource exchange during tar-pit engagement is strictly asymmetric:

Resource	Defender Cost	Attacker Cost
CPU (tar-pit maintenance)	<1% per 256 flows	Full persist-timer retry cycles
Memory	64 bytes per TarpitSlot	Full TCP connection state
Time to clear	10–30 s automatic	Indeterminate (adversary waits)
Information yielded	None	Attack timing exposed to correlation

3.5 Shifting Ghost: The Honeypot Redirection Doctrine

When the escalation gate (21) is satisfied, the most capable adversary flows are not blocked — they are *redirected*. This is the namesake doctrine of the system.

3.5.1 Mechanism: Dynamic DNAT via nftables

The `sg-nftables` module installs a dynamic DNAT rule into the `ip nat` table's `prerouting` chain:

$$\text{src\_ip} = 185.19.40.1,\ \text{dport} = 22 \longrightarrow \text{DNAT to } 10.1.1.100 : 2222$$

This rule is installed and removed at runtime by the Rust enforcement thread, without restarting `nftables`. The adversary's connection is transparently diverted to an isolated Podman container running SSH, HTTP, and other service honeypots.

3.5.2 The Intelligence Inversion

The honeypot achieves something no firewall can: it *reverses the intelligence asymmetry.* While the attacker believes they are engaging the target, they are in fact:

- Revealing their TTPs (tools, techniques, procedures) to the defender's forensic layer
- Exposing their implant's command-and-control protocols through the shell simulation
- Spending exploitation time on a target with no real attack surface
- Generating structured Indicators of Compromise (IOCs) for threat intelligence sharing

Every action within the honeypot is logged and correlated with the flow-level and syscall-level data gathered by the eBPF probes. The resulting incident dossier includes: network flow timeline, process execution trace (from `execve_monitor`), honeypot interaction log, and the full SSA/EKF/entropy score history for the adversary's flow. This dossier is suitable for submission to CERT/CC, intelligence sharing platforms (MISP, TAXII), and legal proceedings.

3.5.3 The Ghost Effect

The adversary who has been redirected to the honeypot is, from their perspective, interacting with the target. They receive plausible responses. They may exfiltrate data that is entirely fabricated. They may execute commands that succeed inside the container and fail silently everywhere else. The intelligence they collect is structurally valid but operationally worthless — or worse, deliberately misleading.

The adversary who discovers the deception faces a worse outcome: they cannot know *when* the deception began. All data collected from this target must be treated as potentially compromised, retroactively. The entire prior reconnaissance effort is invalidated.

IV. Governance, Operational Modes, and the Kill-Switch Doctrine

4.1 The Four Operating Modes

The system's enforcement posture is governed by a four-state mode machine:

Mode	Trigger Condition	Enforcement Posture
Calibration	First 7 days post-deployment	Observe only; no enforcement; baseline gold generation
Automatic	Post-calibration, nominal operations	Full enforcement: tar-pit, honeypot, drop per Eq. (17)
Semi-Manual	>10 blocks/hour or slow-loris/tunnel detection	Alerts generated; no automatic enforcement; operator-required ACK
KillSwitch	Self-compromise detection or admin trigger	All rules flushed; passthrough mode; maximum forensic recording

The KillSwitch mode is noteworthy from a strategic perspective: rather than attempting to maintain enforcement under a detected compromise, the system *maximises forensic visibility*. An adversary who has compromised the IDS itself — a sophisticated capability — triggers a state in which their post-compromise activity is recorded with maximum fidelity, trading enforcement for attribution.

4.2 The Whitelist Immutability Contract

The whitelist (`whitelist.json`) is signed and immutable during normal operation. Any modification triggers automatic escalation to Semi-Manual mode. This constraint defeats a class of attacks in which an adversary, having achieved partial access, attempts to whitelist their own infrastructure to bypass detection. The whitelist modification is auditable, traceable, and operator-acked — it cannot be a silent operation.

4.3 Baseline Gold: The Calibration Chain of Trust

The 7-day calibration phase generates `baseline_gold.json`, encoding the statistical portrait of normal operations: VPIN distribution, MTU profile, PPS mean and variance, EKF steady-state covariance, Hurst baseline, and entropy histograms by time-of-day segment. This file is the root of the system's trust chain:

- Signed with HMAC (SHA-256) and optionally with PGP
- Stored on encrypted storage with restricted access
- Verified on every 24-hour EKF reset cycle
- Any tampering detected at the signature layer triggers Semi-Manual escalation

A defender who maintains custody of `baseline_gold.json` maintains custody of the system's threat model. An adversary who compromises the baseline has not compromised the IDS — they have only triggered a mode transition that alerts a human operator.

V. Strategic Assessment: Why Shifting Ghost is a Structural Liability for Adversaries

5.1 The Five-Failure Cascade

A sophisticated adversary engaging a Shifting Ghost-protected system faces a mandatory failure cascade. There is no attack path that bypasses all five layers simultaneously:

1. **Reconnaissance Fails**: Deceptive fingerprinting and eigenspace manipulation ensure that all pre-attack intelligence is corrupted. The adversary's mental model of the target is wrong before they attack.
2. **Scanning Fails**: The MTU-Track and VPIN mechanisms detect scanning behaviour within two bucket windows (at most 20 seconds). The source is tagged in the eBPF `scan_flag_map` and receives corrupted responses for 60 seconds.
3. **Initial Exploitation Fails**: The P10-enforced, zero-allocation runtime has no heap to overflow, no dynamic allocation to corrupt, and no unbounded loop to spin. Standard memory corruption primitives have no surface to act against.
4. **Sustained Attack Fails**: The Hurst persistence gate ensures that even sophisticated slow-drift attacks that evade per-packet detection are caught over a 30-second to several-minute window. The EKF innovation threshold is calibrated to detect the statistical signature of sustained attack even when individual measurements are sub-threshold.
5. **Post-Exploitation Fails**: eBPF correlation of network flows with `execve` and `write` syscalls ensures that even an attacker who achieves code execution is immediately detected. The kernel-level correlation runs independently of userspace — a compromised application cannot prevent the `execve_monitor` probe from reporting.

5.2 The Rational Adversary Calculus

A rational adversary with accurate knowledge of Shifting Ghost's architecture will perform a cost-benefit analysis before engaging. The system is designed to make this analysis unfavourable:

- **Reconnaissance cost**: High (corrupted returns, wasted analyst time)
- **Attack cost**: High (XDP-level rejection consumes attacker bandwidth with no effect)
- **Expected information value of attack**: Negative (honeypot yields misinformation; IOCs are generated against the attacker)
- **Expected infrastructure risk**: High (eBPF tagging enables persistent tracking; harvested IOCs can be shared with upstream providers)

The system is designed to be *conspicuously expensive to attack once identified.* An adversary who probes the target and receives the characteristic pattern of corrupted fingerprints, zero-window hangs, and implausible response latencies will recognise the signature of a deception-aware, high-assurance defensive system. The rational response is to disengage.

5.3 Limitations and Responsible Caveats

This paper documents a system of considerable complexity. Several limitations must be acknowledged for honest strategic assessment:

- **IPv4 only (current)**: The `FlowKey` and `PacketHeader` types encode 32-bit addresses. IPv6 support requires architectural extension.
- **7-day calibration dependency**: Environments with non-stationary traffic patterns (e.g., extremely variable load) may generate sub-optimal baseline gold files, increasing false positives post-activation.
- **State actor evasion**: Adversaries with knowledge of the Hurst and VPIN thresholds could in principle craft traffic that remains below detection boundaries. This is mitigated by the compound nature of the escalation gate (21), which requires simultaneous threshold crossing on three independent metrics.
- **Legal considerations**: Dynamic DNAT and honeypot operations may carry legal implications in certain jurisdictions. Deployment should be reviewed by legal counsel for compliance with applicable computer crime and data protection regulations.

Conclusion

Shifting Ghost represents a coherent doctrine: that the most effective defence is one that makes attack irrational, not merely difficult. By constructing an immutable runtime substrate from NASA P10 constraints and Rust's ownership model, deploying quantitative detection methods drawn from financial market microstructure theory, maintaining deceptive fingerprinting at the kernel level, and reversing the intelligence asymmetry through a structured honeypot doctrine, the system transforms every engagement into a net loss for the adversary.

The system's sophistication is not in any single component. It is in the *integrated doctrine*: each layer feeds the next. The eBPF ringbuffer feeds the SSA engine. The SSA engine feeds the EKF. The EKF feeds the Bellman scorer. The Bellman scorer arms the fingerprinting probe, the tar-pit, and the honeypot. The honeypot feeds the incident dossier. The incident dossier feeds the baseline update. The loop is closed — and it closes *around the adversary.*

To those defending critical infrastructure, intelligence platforms, and high-assurance compute environments: the question is not whether a system this complex can be built. The question is whether you can afford to operate without one.

Final Doctrine Statement

The adversary who attacks a Shifting Ghost-protected system does not know they are being studied. The adversary who discovers they are being studied does not know when the deception began. The adversary who attempts to disable the system triggers maximum forensic recording. There is no winning move.

End of White Paper No. 2 — WP-SG-002

Shifting Ghost Architecture Team

Quantum-IDS V3.0 — Project Lead Architect Office

This document is subject to review and amendment

as the implementation progresses through Phase gates P1 through P5.